

Celfras Standard Protocol

V2.11.0

Celfras Standard Protocol — Part I: The Standard

This part defines the wire protocol that every Celfras product implements: how bytes are framed on the link, how a request is answered, what the core commands are, and what an implementation has to do to be conforming. It is self-contained. A device or a host can be written against this part alone, without access to any product's source.

Part II is a record, not a requirement. It documents how individual products filled in the parts this standard leaves to the product — identifier maps, optional command groups, layer-specific error codes. Nothing in Part II relaxes anything in Part I.

1. Scope

1.1 What this part defines

- The transport a conforming link runs on (§2).
- The frame format, its COBS encoding, its integrity check, and the receiver's validation rules (§3).
- The request/response model, the status byte, and the core error space (§4).
- Operating modes, and which commands are answered in each (§5).
- The core command set and the standardised command groups (§6, §7, §8).
- The opcode space, what is reserved, and how a product extends it (§9).
- A conformance checklist (§10).

1.2 What it deliberately does not define

Anything a product legitimately differs on: the identifier maps behind the variable and parameter commands (§8), which telemetry field bits carry a real reading (§7.2), the meaning of a tuning identifier (§6.4), and every threshold value a product bakes in. Also, explicitly, the *implementation* of the transport — see §2.

1.3 Roles

Host — the PC-side tool. It issues requests and consumes responses and unsolicited frames.

Device — the product firmware. It answers requests and MAY emit unsolicited frames.

Bridge — an optional relay that sits inline on the same link. It answers the opcodes reserved to it (§9.2) and forwards everything else to the device untouched. A bridge is transparent to a conforming host and device. One rule differs for it, and only one: a bridge **MUST NOT** forward a frame that failed validation, and **MAY** answer such a frame rather than discarding it in silence (§3.7).

Exactly one host and one device participate in a link. A bridge, if present, does not change the frame format in either direction.

1.4 Requirement language

MUST — an absolute requirement of this standard. An implementation that violates one is not conforming. **MUST NOT** — an absolute prohibition of this standard. An implementation that violates one is not conforming. **SHOULD** — a requirement that may be departed from only for a stated reason; a product that departs records it in Part II. **MAY** — genuinely optional.

1.5 Byte order and notation

All multi-byte fields are little-endian and unsigned unless a table says otherwise. Bit 0 is the least significant bit. u8, u16 and u32 denote unsigned integers of that width. Byte values are written in hexadecimal.

2. Transport

The link is asynchronous serial: 8 data bits, no parity, one stop bit, no flow control. The default rate is 1 Mbaud. A product MAY document a different rate, but both ends MUST agree on it out of band — nothing in the protocol negotiates it.

The link is full duplex, and both directions carry the same frame format (§3). The device MAY send an unsolicited frame at any time (§7); a host that cannot tolerate one arriving between its own request and the matching response is not conforming.

The transport backend is not part of this standard. Whether a device serves its UART by DMA, by a per-byte interrupt, or by polling is an implementation choice: nothing on the wire distinguishes them, and a host MUST NOT depend on which one is in use. The reference implementation ships a DMA backend and carries an interrupt backend selectable at build time; the two are byte-for-byte identical on the link, and both are conforming. Statements about ring buffers, transmit queues, interrupt priorities or DMA channels belong to a product's own documentation, never to this standard.

Because the framing in §3.4 is self-synchronising, a receiver MUST recover from a truncated or corrupt frame without an out-of-band reset, and MUST accept frames that arrive back to back with no inter-frame gap. A device MUST NOT require an idle gap, a line break, or a preamble before a frame.

3. Frame format

3.1 The decoded frame

Every frame, in either direction, has this layout once the framing of §3.4 has been removed:

```
[LEN u16 LE][CMD u8][SEQ u8][PAYLOAD ...][CRC16 u16 LE]
```

Field	Width	Meaning
LEN	u16 LE	Number of bytes that follow it: 1 (CMD) + 1 (SEQ) + len(PAYLOAD) + 2 (CRC16).

Field	Width	Meaning
CMD	u8	The opcode. A response carries the same opcode as the request it answers.
SEQ	u8	Correlation token, see §3.5.
PAYLOAD	LEN - 4	Command-specific, and may be empty. For a response it is always [STATUS][DATA...], see §4.
CRC16	u16 LE	Integrity check, see §3.3.

The total decoded frame length is $2 + \text{LEN}$. The shortest legal frame is therefore six bytes: LEN (two) + CMD + SEQ + CRC16 (two), with an empty payload.

3.2 What LEN counts

LEN counts everything **after** the LEN field itself, and that includes the two CRC bytes. It does **not** count itself. A receiver **MUST** check LEN against the length it actually decoded and **MUST** discard any frame where the two disagree — the check is for equality, not for a lower bound.

3.3 The integrity check

CRC16 is **CRC-16/CCITT-FALSE**:

Parameter	Value
Width	16 bits
Polynomial	0x1021
Initial value	0xFFFF
Input reflected	no
Output reflected	no
Final XOR	none

What it covers: the decoded frame from the first byte of LEN through the last byte of PAYLOAD. The two CRC bytes themselves are excluded. It is computed over the decoded frame, never over the encoded bytes of §3.4.

This is the most common interoperability mistake, and two independent things have to be right: the CRC variant, and its coverage. Check the variant first, against these vectors:

Input	CRC16
31 32 33 34 35 36 37 38 39 (ASCII "123456789")	0x29B1
01 00	0x2E3E
(empty)	0xFFFF

0x29B1 over "123456789" is the published check value for this variant. An implementation that produces anything else has the wrong variant — reflection and a final XOR are the usual culprits — and no adjustment of the coverage will fix it. Once the variant is confirmed, check the coverage against the worked frame in §3.6.

3.4 Framing

The decoded frame is COBS-encoded (Consistent Overhead Byte Stuffing), and a single `0x00` byte is appended as the frame delimiter:

```
wire bytes = COBS_encode(decoded frame) + 0x00
```

COBS guarantees the encoded bytes contain no `0x00`, so the delimiter is unambiguous. The delimiter is **not** part of the encoded data and is **not** covered by the CRC.

A receiver **MUST** treat `0x00` as an unconditional frame boundary: accumulate bytes until one arrives, decode what accumulated, then start a new frame. This is what makes the link self-synchronising — after any corruption, the next `0x00` restores frame alignment. A receiver **MUST** discard a zero-length or otherwise undecodable accumulation rather than treating it as a frame.

The encoding is COBS as published by Cheshire and Baker, and the whole of it is stated below. A conforming implementation needs nothing else, and in particular **MUST NOT** be written from the one-line summary "count the non-zero bytes and emit the count": that produces a correct encoder for short frames and a broken one for long ones, and §3.4.1's `0xFF` rule is the difference.

3.4.1 Encoding

The encoder walks the decoded frame and splits it into **groups**. Each group is one of:

- a run of n non-zero bytes, $0 \leq n \leq 253$, terminated by a `0x00` byte of the decoded frame;
- a run of exactly 254 non-zero bytes, terminated by nothing; or
- the final run of n non-zero bytes, $0 \leq n \leq 253$, terminated by the end of the decoded frame.

Each group is emitted as one **code byte** followed by that group's non-zero bytes, in order and unaltered. The code byte is $n + 1$, so it lies in `0x01-0xFF` and is never `0x00`. A terminating `0x00` of the decoded frame is **not** emitted: the code byte stands in for it, which is how the zero is removed from the wire.

A code of `0xFF` therefore means something different from every other value:

- `0x01-0xFE` — code - 1 non-zero bytes follow, **and a `0x00` stood after them** in the decoded frame (or the frame ended there, for the last group).
- `0xFF` — 254 non-zero bytes follow, and **no `0x00` stood after them**. The group ended only because a code byte cannot count higher.

An encoder **MUST** emit a `0xFF` code for every complete run of 254 non-zero bytes, and **MUST** then continue with a fresh group. Splitting a long zero-free run **MUST NOT** introduce a zero that was never in the frame. An encoder **MUST** also emit a code byte for the final group even when that group carries no data bytes: a frame that ends exactly on a 254-byte boundary is encoded as the `0xFF` group followed by a lone `0x01`.

The encoded frame is one byte longer than the decoded frame, plus one further byte for each `0xFF` code — at most $\text{floor}(n / 254)$ of them for an n -byte frame. The delimiter of §3.4 adds one more.

3.4.2 Decoding

A decoder **MUST** process the accumulated bytes — everything received since the previous delimiter, the delimiter itself excluded — as follows:

1. Read one code byte. A code byte of `0x00` **MUST** be rejected: the encoder never produces one, so the accumulation is not a frame.
2. Copy the next code - 1 bytes to the output unaltered. If fewer than code - 1 bytes remain, the frame **MUST** be rejected.
3. If the code was **not** `0xFF` **and** bytes still remain, append one `0x00` byte to the output. If the code was `0xFF`, or nothing remains, append nothing.
4. If bytes remain, return to step 1.

A decoder **MUST** reject the whole frame rather than pass a partial one upward.

Step 3 is the exact inverse of the two code meanings, and both of its conditions carry weight. Appending a zero after a `0xFF` group inserts a byte the sender never sent. Appending one after the *final* group appends a byte that was only ever the end of the frame. Either way the decoded length is wrong by one or more bytes, so the frame fails the LEN and CRC checks of §3.7 and is discarded — which means a wrong decoder shows up as a CRC failure on long frames, not as a decode failure. An implementer chasing that symptom **SHOULD** check step 3 before re-checking the CRC.

3.5 SEQ

SEQ correlates a response with its request.

- A response to a request **MUST** echo that request's SEQ verbatim.
- An unsolicited device-to-host frame — one that answers no request — **MUST** use `SEQ = 0`.
- SEQ is opaque to the device. A device **MUST NOT** interpret it, validate it, or require it to change between requests. Allocating SEQ values, and detecting duplicates or gaps, is entirely the host's business.

A host that pipelines requests **MUST** use SEQ to match responses, and **MUST NOT** assume responses arrive in the order the requests were sent.

Because `SEQ = 0` is how an unsolicited frame identifies itself, a host **SHOULD NOT** issue requests with `SEQ = 0` on a link where streaming (§7) is in use: a host-initiated stop and a device-initiated stop for the same command would otherwise be indistinguishable (§7.2).

3.6 Worked frames

Every byte in this section is computed against the reference implementation, not illustrative.

3.6.1 A six-byte request

A complete CMD_PING request with SEQ = 0x07 and no payload.

Decoded frame — six bytes:

```
04 00 01 07 E7 0D
```

Byte(s)	Value	Field
0-1	04 00	LEN = 4 — CMD + SEQ + zero payload bytes + CRC16
2	01	CMD — the opcode from §6.1
3	07	SEQ — the host's correlation token; the response echoes it
—	—	PAYLOAD — empty for this command
4-5	E7 0D	CRC16 = 0x0DE7, little-endian

The CRC is computed over bytes 0-3, 04 00 01 07 — **including** the two LEN bytes and **excluding** the two CRC bytes. An implementation that starts at CMD instead gets a different value, and its frames are dropped without an error reply (§3.7).

On the wire, after COBS encoding and the delimiter — eight bytes:

```
02 04 05 01 07 E7 0D 00
```

Byte(s)	Value	Meaning
0	02	COBS group code: one non-zero byte follows, and a zero stood here
1	04	LEN low byte
2	05	COBS group code: four non-zero bytes follow
3-6	01 07 E7 0D	CMD, SEQ, CRC16 — carried unchanged
7	00	Frame delimiter — not COBS data, not covered by the CRC

Note what COBS did: the one 0x00 in the decoded frame (the LEN high byte) was removed, and the code byte that replaced it is 0x02 — one data byte, then a zero (§3.4.1). The final code is 0x05: four data bytes, then the end of the frame.

3.6.2 A frame with a 254-byte run

The frame above cannot show the 0xFF code, because six bytes contain no long run. This one can. It is a firmware-image data request, which is where a long zero-free run occurs in practice: image bytes are arbitrary and routinely go hundreds of bytes without a zero. Its opcode belongs to the band reserved to the bridge layer (§9.2) and is defined in Part II — the framing of this section is identical in both bands, and this is the case that exercises it.

The decoded frame is 266 bytes:

Byte(s)	Value	Field
0-1	08 01	LEN = 264
2	E9	CMD
3	09	SEQ
4-7	00 10 00 00	image offset, u32 LE
8-263	01 02 03 ... FD FE 01 02	256 image bytes; byte $8 + i$ holds $(i \bmod 254) + 1$, so no byte of the run is zero
264-265	1A 86	CRC16 = 0x861A, little-endian

Encoded it is 268 bytes, 269 with the delimiter, in five COBS groups:

Encoded byte(s)	Code	Data bytes	What the code says
0-4	05	08 01 E9 09	four non-zero bytes, then a 0x00 — decoded byte 4
5-6	02	10	one non-zero byte, then a 0x00 — decoded byte 6
7	01	—	no data byte, then a 0x00 — decoded byte 7
8-262	FF	01 02 ... FD FE (254 bytes)	254 non-zero bytes and no zero after them
263-267	05	01 02 1A 86	four non-zero bytes, then the end of the frame
268	—	00	Frame delimiter — not COBS data, not covered by the CRC

The first encoded bytes are therefore 05 08 01 E9 09 02 10 01 FF 01 02 03 04 ..., and the last are ... FB FC FD FE 05 01 02 1A 86 00.

The 0xFF at encoded byte 8 is the point of the example. Its 254 data bytes are decoded bytes 8-261; the group that follows resumes at decoded byte 262 with nothing between them. A decoder that inserts a 0x00 there produces 267 decoded bytes instead of 266, and the frame — which arrived intact — fails both the LEN check and the CRC check and is discarded with no reply.

Note the last group too. Its code is 0x05, not 0xFF, and the four bytes it carries are followed by the end of the frame rather than by a zero: decoded byte 265 is the CRC's high byte, and nothing follows it. A decoder that appends an implied zero after the final group is wrong in the same way and by the same one byte.

3.7 Receiver validation, in order

A receiver **MUST** apply these checks, in this order, and **MUST** discard the frame on the first failure:

1. COBS decode (§3.4.2) succeeds and yields at least six bytes.
2. The CRC over the decoded frame, excluding its last two bytes, equals the CRC16 those two bytes carry.
3. LEN equals the decoded length minus two.

A frame that fails any of these **MUST** be discarded, and **MUST NOT** be forwarded. What may be *said* about it depends on the role (§1.3).

A device MUST discard it silently, and MUST NOT answer it. The CMD and SEQ of a frame that failed its integrity check cannot be trusted, so there is no opcode to answer under and no token to answer with; a reply would be a guess, addressed to a request that may never have been sent.

A bridge MAY answer [ERR][ERR_BAD_CRC], under the CMD and SEQ it read, and the reference bridge does. Its position is not the device's: a bridge that says nothing about a frame it refused to forward is indistinguishable from a device that received the frame and ignored it, so answering is what attributes the damage to the link the bridge can see. The same reasoning extends to an implementation whose only role is recovery — a bootloader — where silence is indistinguishable from a board that never started at all.

A host MUST tolerate both outcomes: no answer, and an [ERR][ERR_BAD_CRC] whose SEQ it may not recognise.

Only after all three checks pass is the frame *accepted*. Acceptance is what feeds the link-liveness clock of §5.3, whatever the opcode turns out to be.

3.8 Maximum frame size

A device MUST document the largest encoded frame it accepts, delimiter excluded; the value is a product constant and is recorded in Part II.

The limit is a real boundary, not a guideline. A receiver accumulates encoded bytes into a fixed buffer, and bytes past the end of it are dropped, so an over-long frame arrives as a truncated one and is discarded by §3.7 — silently, with no error reply. A host that sends one observes nothing at all, which is the same thing it observes from a dead link.

- A host MUST NOT send a frame longer than the device's documented limit.
- A host MUST NOT infer the limit by experiment: a frame that was answered proves only that that frame fit.
- A device MUST accept every frame that the command groups it implements can require of it. A device MUST NOT document a limit that excludes a command it answers.

4. Requests, responses and errors

4.1 The model

The protocol is request/response, with an unsolicited side channel.

- Every request the device processes produces **exactly one** response frame, carrying the same CMD and the same SEQ.
- A device MUST NOT answer a request twice, and MUST NOT leave a processed request unanswered — with one exception, the silent-drop mode of §5.2.
- Unsolicited device-to-host frames (§7) carry SEQ = 0 and answer nothing.
- The device is not required to process requests concurrently, and a host MUST NOT assume it does.

4.2 The status byte

A response payload is always [STATUS u8][DATA ...].

STATUS	Meaning
0x00	OK. DATA is the command's result, and may be empty.
0x01	ERR. DATA[0] is an error code (§4.3). Further DATA bytes are permitted and are error-code specific.
0x02	Reserved. No conforming device emits it.

A host **MUST** treat any status other than 0x00 as a failure, and **MUST NOT** desynchronise on an unrecognised status value: consume the frame, report the failure, carry on. Status 0x02 in particular is reserved rather than free — an earlier revision of this protocol declared it as a second error status — so a receiver has to tolerate it arriving even though nothing sends it.

A device **MUST** send [0x01][error code] rather than a bare [0x01]: a host cannot distinguish a bare error from a truncated one.

4.3 The core error space

Error codes 0x00 to 0x0F are the **core space**. It is defined here and nowhere else, and it contains exactly these codes:

Code	Name
0x00	ERR_OK
0x01	ERR_BAD_LEN
0x02	ERR_BAD_CRC
0x03	ERR_BAD_ARGS
0x04	ERR_NOT_READY
0x05	ERR_UNKNOWN

Each has exactly one meaning, and this text is that meaning.

- **ERR_OK (0x00)** — no error. It exists so that zero means the same thing in the code space as it does in the status byte, and it is **never sent**: a device reports success with status 0x00 and no error code at all. [0x01] [0x00] — an error status carrying ERR_OK — is a malformed response, and a host **MUST** treat it as a failure rather than as success.
- **ERR_BAD_LEN (0x01)** — the payload was shorter than the command requires. Decided before any value in it was examined.
- **ERR_BAD_CRC (0x02)** — a frame failed its integrity check, and the layer that received it is reporting that fact instead of discarding it in silence. A device **MUST NOT** send this code; a bridge or a bootloader **MAY** (§3.7).
- **ERR_BAD_ARGS (0x03)** — the payload was long enough, but something in it was out of range, unknown, or not writable. It is also the answer to an opcode the device does not act on (§9.1).

- **ERR_NOT_READY (0x04)** — the command is recognised and its arguments are valid, but the device's current state does not permit it. The same request, unchanged, may succeed later. It is the answer for a session that is already open (§6.4), a mode that has not been entered, or any other precondition the host has not met.
- **ERR_UNKNOWN (0x05)** — the opcode is not recognised. Optional, and interchangeable with ERR_BAD_ARGS for that condition; see §9.1.

A product **MUST NOT** define a new code in 0x00-0x0F, **MUST NOT** give an existing one a second meaning, and **MUST NOT** use one for a condition other than the one defined above. These are the codes a host can interpret without knowing what it is talking to.

Note the shape of the two most common ones. ERR_BAD_LEN means the payload was too short for the command to be executed at all, decided before any value in it was examined. ERR_BAD_ARGS means the payload was long enough but something in it was out of range, unknown, or not writable. A device **MUST NOT** answer ERR_BAD_ARGS for a short payload or ERR_BAD_LEN for a bad value — a host uses the difference to tell a mis-built frame from a mis-chosen value.

A command's payload length is a **minimum**, and only LEN is checked for equality (§3.2). A device **MUST NOT** answer ERR_BAD_LEN because a payload was *longer* than the command requires; it **MUST** read the bytes the command defines and ignore the rest. A host **MUST NOT** send trailing bytes expecting them to be read, and **MUST NOT** take an OK response as evidence that they were: the same response comes back either way, which is exactly why the excess cannot carry meaning.

4.4 The extension space

Codes 0x10 and above are an **extension space**, and this standard assigns nothing in it. A code there means whatever the layer that sent it says it means, and it is interpretable only once the host knows which layer answered.

This is not hypothetical. The bridge layer (§9.2) uses 0x11-0x15 for its SWD and core-control operations, 0x20, 0x21 and 0xE1 for flash operations, and 0x30-0x35 for its own firmware update. Error codes and opcodes are separate spaces: the 0xE1 named here is an error code, and the fact that 0xE1 also falls inside the bridge's reserved *opcode* band (§9.2) says nothing about it — where the byte sits in the frame is what decides which space it belongs to. The first block is worth naming precisely: 0x14 and 0x15 are core halt and core resume, which sit beside the SWD errors without being SWD errors, and reading them as transport faults has cost bench time. Those meanings are the bridge's — the blocks named here are the whole of that layer's allocation, and the bridge appendix of Part II carries the commands that raise them — and they say nothing about what a different layer might assign to the same numbers. A device and a bridge on one link can both use 0x30 for unrelated things without conflict, because the frame says which of them answered.

Rules a product **MUST** follow when it needs its own error codes:

1. Allocate at 0x10 or above. Never in the core space.
2. Document the allocation in Part II, against the layer that owns it.
3. Assume nothing about numbers outside your own layer — a code is not free just because some other layer does not use it.

5. Operating modes

5.1 The modes

A device is always in exactly one operating mode. `CMD_SET_MODE` (§6.1) is what changes it.

Value	Name
0x00	OPMODE_ISP
0x01	OPMODE_NORMAL
0x02	OPMODE_DEBUG
0x03	OPMODE_TUNING
0x04	OPMODE_TEST

The standard assigns the values and fixes what each mode is *for*. What a mode does inside a product — which subsystems it starts, suspends or ignores — is product-defined and recorded in Part II.

- **OPMODE_ISP** — selects the product's in-system-programming path. A device that implements it MAY stop acting on its ordinary inputs while in it, and MAY return itself to `OPMODE_NORMAL` on its own if nothing further arrives.
- **OPMODE_NORMAL** — the boot default, and the quiet one. See §5.2.
- **OPMODE_DEBUG** — every command the device implements is answered.
- **OPMODE_TUNING** — reserved for host-driven calibration (§6.4).
- **OPMODE_TEST** — reserved for a product's own production test.

A device **MUST** boot into `OPMODE_NORMAL`. A device **MUST** answer `CMD_SET_MODE` with `ERR_BAD_ARGS` for a mode value it does not implement, rather than ignoring it or selecting a neighbouring mode.

A reserved mode that a device accepts without acting on is still acknowledged as OK — the acknowledgement means "the value is recognised", not "the mode did something".

5.2 What **NORMAL** answers

`OPMODE_NORMAL` is the boot default, and it is deliberately quiet: it isolates the device's own timing from host traffic until a host asks for it.

In `OPMODE_NORMAL` a device **MUST** process, and answer, exactly these commands: `CMD_PING`, `CMD_INFO`, `CMD_GET_VERSION`, `CMD_SET_MODE`, `CMD_DBG_ONLINE` and `CMD_RESET` (§6.1, §6.2). Every other opcode — assigned, unassigned or unimplemented alike — **MUST** be **silently dropped**: no response at all, not even an error, and no action taken.

Those six are the normative set; no other list of them anywhere is authoritative.

This is the one place in this standard where a well-formed request goes unanswered, and both ends have to understand it the same way. A host that gets no answer in `OPMODE_NORMAL` has not found a broken device; it has found a device nobody has asked to leave the boot mode. `CMD_SET_MODE` is processed in every mode, and is the only way out.

CMD_PING, CMD_INFO and CMD_GET_VERSION are answered in every mode by design: identifying a device and checking that it is alive MUST NOT require changing its state.

5.3 Link liveness

While in any mode other than OPMODE_NORMAL, a device MUST return itself to OPMODE_NORMAL if no frame is accepted (§3.7) for longer than the link-liveness interval, ONLINE_TIMEOUT_MS. The timer is reset by **any** accepted frame, not by one specific opcode.

A host that intends to stay out of OPMODE_NORMAL MUST therefore send *something* within every such interval. CMD_DBG_ONLINE exists as the cheapest choice — it takes no payload and does nothing else — but any accepted frame serves.

The purpose is not liveness reporting. It is that a mode other than the boot default may have suspended some of the device's own protective behaviour, and a pulled cable must not leave it suspended. A conforming device MUST make the revert unconditional: it cannot depend on the host acknowledging anything, because the failure being defended against is the host being gone.

The interval's value is a product constant, reported in Part II.

6. The core command set

The tables in this section are generated from the authoritative opcode definitions. Request and response shapes are given in the tables; the prose gives the rules around them.

6.1 Identity, mode and liveness

ID	Name	Request	Response	Notes
0x01	CMD_PING	[]	[OK][<i>"PONG"</i>]	
0x02	CMD_INFO	[]	[OK][id string, contains "CFS"]	
0x03	CMD_SET_MODE	[op_mode u8]	[OK] or [ERR][ERR_BAD_ARGS]	
0x04	CMD_DBG_ONLINE	[]	[OK]	liveness ping, processed in any op_mode
0x05	CMD_GET_VERSION	[sel u8]	[OK][sel][major][minor][patch]	

Every command in this group MUST be implemented by every conforming device, and every one of them is answered in every operating mode.

- **CMD_PING** — the liveness probe. Its response payload is fixed text; a host SHOULD check the status byte rather than parse the text.
- **CMD_INFO** — a human-readable identity string. It is free text, and its content is product-defined. A host MUST NOT parse it to make a decision; CMD_GET_VERSION exists for that.
- **CMD_SET_MODE** — see §5.
- **CMD_DBG_ONLINE** — a no-op whose only effect is resetting the liveness timer of §5.3.
- **CMD_GET_VERSION** — the machine-readable identity. One opcode answers more than one question, through a leading selector byte, and the response echoes the selector so that a reply is

self-describing: readable out of a log, and matchable by a host that pipelined both requests.

The request is [sel u8] and the response is [OK][sel u8][major u8][minor u8][patch u8], with the selector repeated at DATA[0]. This standard defines two selectors:

Value	Name	Meaning
0x00	VER_SEL_FW	the firmware build on this board
0x01	VER_SEL_CMD_SET	the opcode table it implements

VER_SEL_FW reports the version of the firmware image that answered — a product identity, and not comparable across products. VER_SEL_CMD_SET reports the command-set version of §9.3, which every implementation of this standard carries and which is comparable across all of them. Both are the same MAJOR.MINOR.PATCH shape, and neither can be derived from the other.

Selector values 0x02 and above are unassigned by this standard. A device **MUST** answer ERR_BAD_ARGS for a selector it does not implement, and **MUST NOT** answer with a different selector's value — a host that pipelined two requests has only the echoed selector to tell the replies apart. Every conforming device **MUST** implement both selectors above. Selector numbering is per opcode and is not shared: the bridge layer has its own version command (§9.2, Part II) with its own selector space, and the same selector value there means something else.

A host **SHOULD** read the command-set version before relying on any opcode outside this group.

6.2 Device control

ID	Name	Request	Response	Notes
0x10	CMD_RESET	[]	[OK] ["SYSTEM RESET"], then NVIC_SystemReset()	
0x11	CMD_ACTIVATE	[]	[OK] (stub)	
0x12	CMD_DEACTIVATE	[]	[OK] (stub)	
0x13	CMD_START_HEATING	[]	[OK][3 bytes] (stub)	
0x14	CMD_STOP_HEATING	[]	[OK][3 bytes] (stub)	

CMD_RESET is the command in this group required of every device, and it is answered in every mode. A device **MUST** send its response **before** resetting, and **MUST** allow that response to leave the transmit path first; no frame follows the reset. A host **MUST** treat the link as re-initialised afterwards — the device comes back in OPMODE_NORMAL (§5.1).

The remaining opcodes in this group are the standardised places for a product's own activation and heating control. A product that does not implement one **MUST** answer it per §9.1 rather than acknowledging it, and **MUST NOT** reuse one of these opcodes for an unrelated function.

A known deviation, stated rather than smoothed over. The reference firmware does not meet that rule today: CMD_ACTIVATE, CMD_DEACTIVATE, CMD_START_HEATING and CMD_STOP_HEATING answer [OK] whether or not the product acts on them, which is why the table above marks each of them a stub. The rule stands and a new implementation **MUST NOT** copy the behaviour; the consequence while it lasts is

that an [OK] from those four opcodes is not evidence that anything happened, and a host MUST NOT read it as such. Part II records which products are affected.

6.3 Register access

ID	Name	Request	Response	Notes
0x20	CMD_REG_WRITE8	[reg_addr u8][reg_data u8]	[OK][resp_data u8]	
0x21	CMD_REG_READ8	[reg_addr u8]	[OK][resp_data u8]	
0x22	CMD_REG_WRITE16	[reg_addr u8][reg_data u16]	[OK][resp_data u16]	
0x23	CMD_REG_READ16	[reg_addr u8]	[OK][resp_data u16]	
0x24	CMD_REG_BURST_READ16	[reg_addr u8]	[OK][resp_data u16]	

An optional group, for products that expose a peripheral or companion-chip register file. Register addresses and their contents are entirely product-defined. A product with no register file MUST answer this group per §9.1.

6.4 Tuning

ID	Name	Request	Response	Notes
0x70	CMD_TUNING_START	[tuning_id u8][duration_ms u32][log_fields_mask u16][report_period_ms u16]	[OK] or [ERR] [ERR_BAD_ARGS/ERR_BAD_LEN]	
0x71	CMD_TUNING_REPORT_FRAME		[OK][tick_ms u32][field u16]... (one u16 per set bit in log_fields_mask, LSB first)	(fw->host only, seq=0) same shape family as CMD_LOG_BURST_FRAME
0x72	CMD_TUNING_END	[]	[OK]	host-initiated (echoes req seq) or fw-initiated on duration timeout (seq=0), same idempotent shape as CMD_LOG_BURST_STOP
0x73	CMD_SET_TUNING_PAR	[tuning_id u8][coil_idx u8][heating_mode u8][heater_type u8][step u8][value u16 LE]	[OK] or [ERR] [ERR_BAD_ARGS/ERR_BAD_LEN]	no session required
0x74	CMD_GET_TUNING_PAR	[tuning_id u8][coil_idx u8][heating_mode u8][heater_type u8][step u8]	[OK][value u16 LE] or [ERR] [ERR_BAD_ARGS/ERR_BAD_LEN]	no session required

An optional group for host-driven calibration, in which the device runs its real production control algorithm under host-controlled timing so that a bench can measure it. The standard fixes the **shape**, not the content:

- CMD_TUNING_START opens a session; a second one while a session is active MUST be refused with ERR_NOT_READY.
- While a session is active, the device emits report frames unsolicited at the requested cadence, with SEQ = 0, in the field layout of §7.2.
- CMD_TUNING_END closes a session and is **dual-purpose**, exactly as CMD_LOG_BURST_STOP is (§7.2): host-initiated, it echoes the request's SEQ and is idempotent; device-initiated — on the session's

duration elapsing, or because the device ended the run itself — it arrives with $SEQ = 0$. A host **MUST** handle both arrival paths as the same session-ended event.

- The device **MAY** end a session early for its own reasons. A host **MUST NOT** assume a session runs for its full requested duration, and **MUST NOT** assume every session yields at least one report frame.
- The parameter commands in this group address a calibration table by a tuple of indices. Every index **MUST** be bounds-checked against the real dimension of the table it addresses, and a violation **MUST** be `ERR_BAD_ARGS` — never a clamp, and never a silent write to a neighbouring cell.

Which tuning identifiers exist, what each one sweeps, what the index tuple means for each, and which suspensions of a product's own protective behaviour a session implies, are all product-defined and recorded in Part II.

6.5 Display

ID	Name	Request	Response	Notes
0x60	CMD_DISP_ON	[]	[OK]	
0x61	CMD_DISP_OFF	[]	[OK]	

An optional group. A product with no host-controllable display answers it per §9.1.

7. Log and telemetry streaming

Both mechanisms in this section send device-to-host frames that answer no request. All of them carry $SEQ = 0$ (§3.5).

Both groups are **optional**. A product that implements neither **MUST** answer their opcodes per §9.1; nothing else in this standard depends on them.

An unsolicited frame is shaped exactly like a response. Answering no request does not change its payload: it is `[STATUS u8][DATA ...]` as §4.2 requires, with `STATUS = 0x00`, and the frame's own content begins at `DATA[0]` — that is, at payload byte 1, not payload byte 0. This holds for every frame in this section and for the tuning report frames of §6.4. A host that reads the first payload byte as content is off by one on every sample it ever takes, and the error is silent, because a status byte of `0x00` is a plausible first data byte. The generated tables spell the shape out, [OK] first.

7.1 Text log

ID	Name	Request	Response	Notes
0x30	CMD_LOG_FRAME		[OK][text bytes]	(fw->host only)
0x31	CMD_LOG_START	[]	[OK]	
0x32	CMD_LOG_STOP	[] (host) or (fw->host, unsolicited)	[OK]	

A pull-based handshake for free-text diagnostics, shaped so that the device's log output cannot flood the link:

1. The host sends `CMD_LOG_START`.
2. The device **MUST** send its acknowledgement **before** it arms logging, and **MUST** queue the acknowledgement ahead of anything that follows it, so the host always sees the ack before any log content. The ordering is load-bearing: a device that arms first can emit a log line while the host is still waiting for its response, and the host will parse that line as the response it asked for.
3. The device sends the next log line as one `CMD_LOG_FRAME` carrying the text, followed by one `CMD_LOG_STOP`.
4. The host re-issues `CMD_LOG_START` for the next line.

A host **MUST** tolerate `CMD_LOG_STOP` arriving with no `CMD_LOG_FRAME` before it: that means the device had nothing to say. A host **MUST NOT** assume the text is NUL-terminated, and **MUST** take the payload length as authoritative.

A host **MUST** also tolerate a device that keeps emitting frames without a further `CMD_LOG_START` — the disarm step is what bounds the stream to one line, and a product **MAY** deliberately leave it armed for continuous telemetry. A host that needs the stream to stop **MUST** have a way to disarm it: `CMD_SET_MODE` selecting `OPMODE_NORMAL` always does, and a product **MAY** also expose logging as a parameter slot (§8).

Text logging is for diagnostics. Anything sampled at a fixed cadence belongs in §7.2 instead — text formatting and parsing at those rates is exactly the cost the binary path exists to avoid.

7.2 Binary telemetry bursts

ID	Name	Request	Response	Notes
0x40	<code>CMD_LOG_BURST_START</code>	[fields_mask u16] [period_ms u16] [duration_ms u32]	[OK] or [ERR] [ERR_BAD_ARGS/ERR_BAD_LEN]	
0x41	<code>CMD_LOG_BURST_FRAME</code>		[OK][tick_ms u32][field u16]... (one u16 per set bit in fields_mask, LSB first)	(fw->host only, seq=0)
0x42	<code>CMD_LOG_BURST_STOP</code>	[]	[OK][total_frame_cnt u16]	host-initiated (echoes req seq) or fw-initiated on duration timeout (seq=0)

A fixed-width binary stream for time-series measurement.

Starting a session. `CMD_LOG_BURST_START` takes a field bitmask, a report period and a duration. The period **MUST** lie within `[BURST_PERIOD_MS_MIN, BURST_PERIOD_MS_MAX]`, and the duration, if non-zero, within `[BURST_DURATION_MS_MIN, BURST_DURATION_MS_MAX]`; a value outside either range **MUST** be refused with `ERR_BAD_ARGS`, not clamped.

These are protocol constants of this standard, identical for every product, and this is where their values live:

Name	Value
BURST_DURATION_MS_INFINITE	0
BURST_DURATION_MS_MAX	1000000
BURST_DURATION_MS_MIN	100
BURST_MAX_FIELDS	16
BURST_PERIOD_MS_MAX	1000
BURST_PERIOD_MS_MIN	10

BURST_DURATION_MS_INFINITE is a reserved duration value, not a member of the duration range: it means *no* duration, and a session started with it runs until the host stops it (**Stopping**, below). It is the one duration below BURST_DURATION_MS_MIN that a device MUST accept. BURST_MAX_FIELDS is how many field bits the mask can carry, which is why the mask is a u16.

Period and duration are both in milliseconds.

The field mask selects signals from the standard LOG_FIELD_* family, one bit each. The bit assignments are part of this standard:

Bit	Name
0x01	LOG_FIELD_VDD
0x02	LOG_FIELD_VAT
0x04	LOG_FIELD_IAT
0x08	LOG_FIELD_PWR
0x10	LOG_FIELD_DUTY
0x20	LOG_FIELD_PROT
0x40	LOG_FIELD_RAT

Each bit names a quantity, and the quantity is what the standard fixes:

- LOG_FIELD_VDD — the device's own supply rail.
- LOG_FIELD_VAT — the voltage across the driven load.
- LOG_FIELD_IAT — the current through it.
- LOG_FIELD_PWR — the power delivered to it.
- LOG_FIELD_DUTY — the duty cycle of the drive.
- LOG_FIELD_PROT — the device's protection status, as a bitfield.
- LOG_FIELD_RAT — the load resistance the device computed.

Every field is carried as a u16. Two things are product-defined and recorded in Part II: the **scale, unit and derivation** behind each field — a host that reads a value without knowing the product's scale has a number, not a measurement — and **which bits the product has wired** to a real reading at all.

A device MUST accept a bit it has not wired, without error, and MUST report that field as zero. Bits above the highest assigned one are unassigned by this standard, and a device MUST treat an

unassigned bit exactly as it treats an unwired one: accepted, reported as zero. Both rules exist so that a host built against a later product does not fail against an older one.

Frame layout. Each CMD_LOG_BURST_FRAME payload is [STATUS][tick_ms u32][field u16] ...: the status byte of §4.2, always 0x00; then a u32 timestamp in milliseconds; then one u16 per bit **set in the requested mask**, in ascending bit order. The order is fixed by the bit numbers and is not the host's to choose; a bit that is not set contributes no u16 at all. The timestamp MUST NOT decrease within a session and its zero point MUST NOT change during one; where that zero point sits is product-defined. The mask is not repeated in the frame: the host MUST decode using the mask it sent. A device MUST NOT reorder the values, and MUST NOT omit one.

Stopping. CMD_LOG_BURST_STOP is dual-purpose, and this is the shape the tuning group reuses (§6.4):

- *Host-initiated* — a request. The device stops any active session and answers with the request's own SEQ. It MUST be idempotent: stopping a session that is not running is OK, not an error. This is the only way to end an infinite-duration session, and it also cancels a finite one early.
- *Device-initiated* — unsolicited, SEQ = 0, sent when a non-zero duration elapses.

Both carry the same payload. A host MUST resolve its session state from either.

Concurrency. Sampling MUST NOT block command dispatch: ordinary traffic, including the host's own stop request, has to keep flowing while a session is active. A device that runs its sweep inside the command handler cannot be stopped, and is not conforming.

8. Variables and parameters

ID	Name	Request	Response	Notes
0x50	CMD_VAR8_SET	[var_id u8][value u8]	[OK]	
0x51	CMD_VAR8_GET	[var_id u8]	[OK][value u8]	
0x52	CMD_VAR16_SET	[var_id u8][value u16 LE]	[OK]	
0x53	CMD_VAR16_GET	[var_id u8]	[OK][value u16 LE]	
0x54	CMD_VAR32_SET	[var_id u8][value u32 LE]	[OK]	
0x55	CMD_VAR32_GET	[var_id u8]	[OK][value u32 LE]	
0x56	CMD_PAR8_SET	[par_id u8][value u8]	[OK]	
0x57	CMD_PAR8_GET	[par_id u8]	[OK][value u8]	
0x58	CMD_PAR16_SET	[par_id u8][value u16 LE]	[OK]	
0x59	CMD_PAR16_GET	[par_id u8]	[OK][value u16 LE]	
0x5A	CMD_PAR32_SET	[par_id u8][value u32 LE]	[OK]	
0x5B	CMD_PAR32_GET	[par_id u8]	[OK][value u32 LE]	

Two families of commands give indexed access to device state, in three widths each. The distinction is what they reach:

- **CMD_VAR*** — the **variable** map: live runtime state. Mostly a read-only view of what the firmware is actually doing (state, measurements, timers, gauges) rather than a second copy of it, plus a small writable group of host-supplied inputs that stand in for something the device would otherwise read from its own hardware.
- **CMD_PAR*** — the **parameter** map: the device's configuration and threshold store.

This group is **optional**. A product that implements neither family MUST answer all twelve opcodes per §9.1. A product MAY implement one family without the other, and MAY implement one width without the others — but a width that is implemented MUST implement both its SET and its GET, since a value that can be written and not read back cannot be checked.

Both families share one shape. SET takes [id u8][value]; GET takes [id u8] and answers [OK][value]; and the value is a u8, u16 or u32, little-endian, according to the command's width. The three widths are three separate identifier spaces — id 3 of the 8-bit space and id 3 of the 16-bit space are unrelated slots.

Required behaviour:

- An id at or beyond the width's table size MUST be answered ERR_BAD_ARGS, before any dispatch is attempted.
- A SET to a read-only slot MUST be answered ERR_BAD_ARGS. It MUST NOT store the value, and MUST NOT answer OK — a host cannot tell a silently ignored write from a successful one.
- A conforming **device** MUST document the range it accepts for each writable slot. The range is the device's own statement about itself: a slot that accepts every value its width can hold documents that, and a slot that accepts less documents what it accepts. This standard does not prescribe where that documentation lives or what form it takes — a datasheet, a product manual, or an appendix of the shape Part II uses — only that the device's own statement of its ranges exists and is the one a host uses. Limits a host tool keeps in its own copy of the identifier map are that tool's input affordance, adjustable by whoever runs it, and are not the device's range — a host MUST NOT treat them as one.
- A SET with a value the slot does not accept MUST be answered ERR_BAD_ARGS. A device MUST NOT mask or clamp the value into range, and MUST NOT answer OK to a value it then discards: a host asking for something impossible has a bug, and every one of those three outcomes hands it plausible-looking readings instead of the error that would surface it.
- A GET of a slot, issued after a successful SET of that same slot, MUST return the value that was set, unless the device itself has changed it in the meantime. This is the observable form of the rule, and it is the only form a host can check: where the value is kept is the device's business, but a write that cannot be read back is indistinguishable from a write that was discarded. A slot that cannot meet this MUST reject SET with ERR_BAD_ARGS and MUST be declared read-only.

Those last two are one rule seen from the wire, and it is the whole of what a host can observe: **after a SET answered OK, a GET of that slot returns what was set; and a device that will not take a value says so instead of answering OK**. There is no conforming third outcome — no OK that means the device kept something else, and none that means it kept nothing.

The failure this forbids is easy to build by accident, and it is worth naming because it is not a protocol mistake at all. Where a slot's setter refuses a value inside the device but has no way to report the refusal

back to the command handler — a setter that returns `void` is the usual shape — the write is dropped and the handler, having nothing to report, answers OK. Nothing on the wire contradicts that until a GET disagrees with the last value written, and a host that never re-reads the slot will not find out at all. The refusal has to reach the layer that composes the response, and that layer has to send `ERR_BAD_ARGS`.

8.1 The identifier maps are product-defined, and this is load-bearing

This standard defines the commands, the widths and the error behaviour. It does **not** define which slot number means which quantity. Those maps are assigned per product, and are recorded in Part II.

They are **not** portable between products, and the protocol cannot defend against using the wrong one. A slot number valid on one product is very likely valid on another and addresses a completely different field — so the device performs the wrong write and answers **OK**. There is no error to observe, at any layer. Therefore:

- A host **MUST** establish which product, and which command-set version, it is talking to (`CMD_INFO`, `CMD_GET_VERSION`) before issuing any VAR/PAR access, and **MUST** use that product's own map.
- A product **MUST** assign new slots by appending. Renumbering an existing slot changes what an unchanged host writes, and is a MAJOR command-set change (§9.3).

8.2 Where two opcodes reach one field, they **MUST** agree

This standard defines more than one route to a device's configurable state: the parameter family above, and, on a product that implements the tuning group, that group's own parameter commands (§6.4). A product **MAY** expose the same underlying field through more than one of them.

Where it does, those opcodes **MUST** agree about that field. They **MUST** agree on whether it is writable at all, and they **MUST** accept the same values for it.

Without that rule a product can answer two different things about one field in one build: a write accepted and answered OK through one opcode, and the identical write refused with `ERR_BAD_ARGS` through another. A host has no way to resolve the contradiction. Neither answer is wrong on its own terms — each handler is applying a rule the other does not know about — and nothing in the frame says which one describes the device, so the host is left to decide whether the field took the value by re-reading it through a third route.

Writability is a property of the field, not of the opcode a host happened to reach it through, and a host is entitled to establish it once and rely on it everywhere. A product that means a field to be reachable by only one opcode expresses that by not mapping it into the other, rather than by mapping it and refusing it there.

9. Opcode space, reservations and evolution

9.1 Unassigned and unimplemented opcodes

Outside `OPMODE_NORMAL`, a device **MUST** answer **every** opcode it does not act on with `[ERR]` `[ERR_BAD_ARGS]`. That covers both cases equally: an opcode this standard has not assigned, and one it

has assigned that this product does not implement. Silence is not a conforming answer — a host has to be able to tell an unsupported command from a dead link.

The single exception is the silent-drop behaviour of `OPMODE_NORMAL` (§5.2).

`ERR_UNKNOWN` exists in the core error space for this purpose, and a device **MAY** use it where it can distinguish an unrecognised opcode from a bad argument. A host **MUST** accept either code as "this device will not do that".

9.2 Band reservation

The opcode byte is split into two bands, and the split is what lets one physical link carry two conversations:

Band	Owner
0x00-0xBF	The device (the product firmware). New commands grow upward within it.
0xC0-0xFF	Reserved to the bridge layer. A product MUST NOT define a command here.

A bridge routes on the opcode byte alone: at `0xC0` and above it answers the frame itself, and below `0xC0` it forwards the original encoded bytes to the device untouched. A product command defined at `0xC0` or above would therefore be unreachable whenever a bridge is inline — and the failure is silent. There is no error to observe: the bridge consumes the frame and answers it, the device never sees it, and the host gets a plausible reply from the wrong device.

The reservation is one band for the whole layer, not one board's band. Every bridge implementation declares the same table.

Within the device's own band, opcodes are grouped, and a product **SHOULD** place a new command in the group that matches its function:

Base	Group
0x00	Identity, mode, liveness (§6.1)
0x10	Device control (§6.2)
0x20	Register access (§6.3)
0x30	Text log (§7.1)
0x40	Binary telemetry (§7.2)
0x50	Variables and parameters (§8)
0x60	Display (§6.5)
0x70	Tuning (§6.4)

9.3 Versioning

The **command-set version** is a `MAJOR.MINOR.PATCH` triple naming this standard's opcode map — the whole map, both bands. It is not any firmware's build number, and it moves independently of one.

- **MAJOR** changes when an existing opcode changes its number or its meaning. A host built against the older table would otherwise reach a different handler and get a plausible answer, which is precisely the failure the version exists to make detectable.
- **MINOR** changes when opcodes are added and every existing one still means what it did.
- **PATCH** is for changes with no effect on the wire.

Two different rules use this number, and conflating them is how a real system gets refused for a difference that was never a fault.

Release bookkeeping — a property of a build. Within one release of one implementation, every copy of the command-set version constant **MUST** carry the same value, and every copy **MUST** move together when it changes. The number is duplicated across firmwares and host tools in more than one repository, so the copies have to be counted rather than assumed; Part II lists them for the products it covers. A release that ships two different values for its own command-set version is mis-built. Nothing about this rule is observable on the wire.

Wire compatibility — a property of a link. Two implementations on one link **MAY** report different command-set versions, and a difference is not by itself a fault. A device reporting an older version is stating truthfully which opcode table it was built against, which is the fact worth having. A host:

- **SHOULD** read the far end's version (CMD_GET_VERSION with VER_SEL_CMD_SET, §6.1) before relying on any opcode outside §6.1;
- **SHOULD** refuse to proceed on a MAJOR difference, because an opcode may have changed number or meaning underneath it and the answers will look plausible;
- **MAY** proceed on a MINOR or PATCH difference, and **SHOULD** then confine itself to the opcodes defined by the lower of the two versions. MINOR is additive by definition, so the older end has fewer commands, not different ones.

10. Conformance checklist

A conforming **device** answers yes to every item below that applies to it.

When an item does not apply. Five command groups are optional: register access (§6.3), tuning (§6.4), display (§6.5), streaming (§7) and variables and parameters (§8). An item that tests a group the device does not implement is answered **N/A**, and a device with N/A items is still conforming — provided item 25 holds for that group's opcodes, which is what makes "not implemented" something a host can observe rather than something it has to be told. No item under Transport and framing, Requests and responses, Modes, or items 22, 23, 24, 25, 26 and 27 is ever N/A: those are required of every device.

Transport and framing

Item	Requirement	See
1	The serial link is 8 data bits, no parity, one stop bit, no flow control, at the agreed rate.	2
2	No inter-frame gap, break or preamble is required of the host, and back-to-back frames are accepted.	2

Item	Requirement	See
3	Every frame is COBS-encoded with a trailing 0x00 delimiter, and 0x00 is treated as an unconditional frame boundary on receive.	3.4
4	On encode, every complete 254-byte zero-free run is emitted with a 0xFF code, no 0x00 is introduced where none stood, and the final group carries a code byte even when it has no data.	3.4.1
5	On decode, a 0x00 is restored after every group whose code was not 0xFF, and after no other group — including the last one in the frame.	3.4.2
6	LEN counts CMD + SEQ + payload + CRC, and is checked for equality against the decoded length.	3.2
7	The CRC is CRC-16/CCITT-FALSE and reproduces 0x29B1 over "123456789".	3.3
8	The CRC covers the decoded frame from the first LEN byte through the last payload byte, and excludes the CRC bytes themselves.	3.3
9	A frame failing COBS decode, the CRC check or the LEN check is discarded silently, with no response.	3.7
10	The largest accepted encoded frame is documented, and a longer one is discarded silently like any other frame that cannot be validated.	3.8

Requests and responses

Item	Requirement	See
11	Each processed request produces exactly one response, echoing its CMD and its SEQ.	4.1
12	Unsolicited frames carry SEQ = 0.	3.5
13	SEQ is never interpreted or validated by the device.	3.5
14	Every response payload begins with a status byte, and an error response carries an error code in DATA[0].	4.2
15	Each core error code is used only for the condition §4.3 defines for it, and any product-specific code is at 0x10 or above.	4.3, 4.4
16	ERR_BAD_LEN is used for a short payload and ERR_BAD_ARGS for a bad value, never interchangeably.	4.3
17	A payload longer than the command requires is accepted and its excess ignored, never answered ERR_BAD_LEN.	4.3

Modes

Item	Requirement	See
18	The device boots into OPMODE_NORMAL.	5.1
19	CMD_SET_MODE is processed in every mode, and an unimplemented mode value is refused with ERR_BAD_ARGS.	5.1
20	In OPMODE_NORMAL, CMD_PING, CMD_INFO, CMD_GET_VERSION, CMD_SET_MODE, CMD_DBG_ONLINE and CMD_RESET are answered, and everything else is dropped silently.	5.2
21	Any non-default mode reverts to OPMODE_NORMAL after ONLINE_TIMEOUT_MS with no accepted frame, unconditionally.	5.3

Commands

Item	Requirement	See
22	Every command in §6.1 is implemented, and answered in every mode.	6.1
23	CMD_GET_VERSION implements both VER_SEL_FW and VER_SEL_CMD_SET, echoes the selector it was given, answers ERR_BAD_ARGS for any other, and reports the version of §9.3 under VER_SEL_CMD_SET.	6.1, 9.3
24	CMD_RESET responds before resetting.	6.2
25	Outside OPMODE_NORMAL, every opcode the device does not act on is answered [ERR][ERR_BAD_ARGS], or [ERR][ERR_UNKNOWN] where the device can tell an unrecognised opcode from a bad argument.	9.1
26	No command is defined at 0xC0 or above.	9.2
27	Every copy of the command-set version constant in the release carries the same value.	9.3

Streaming — N/A for a device that implements neither §7.1 nor §7.2.

Item	Requirement	See
28	Log frames and telemetry frames are emitted with SEQ = 0.	7
29	An unsolicited frame's payload begins with the status byte, and the frame's own content begins at DATA[0].	7, 4.2
30	CMD_LOG_START is acknowledged before logging is armed, and the acknowledgement is queued ahead of any log content.	7.1
31	A burst period or duration outside its permitted range is refused with ERR_BAD_ARGS, not clamped, and BURST_DURATION_MS_INFINITE is accepted.	7.2
32	A burst frame carries one u16 per set mask bit, in ascending bit order, and an unwired or unassigned bit reports zero rather than erroring.	7.2
33	A stop or end command is idempotent, echoes the request's SEQ when host-initiated, and is also emitted unsolicited with SEQ = 0 when the device ends the session itself.	6.4, 7.2
34	Command dispatch keeps working while a streaming session is active.	7.2

Variables and parameters — N/A for a device that implements neither family.

Item	Requirement	See
35	An out-of-range identifier is refused with ERR_BAD_ARGS.	8
36	A write to a read-only slot is refused with ERR_BAD_ARGS and stores nothing.	8
37	The device documents the range it accepts for each writable slot, and a value it does not accept is refused with ERR_BAD_ARGS — never clamped, never masked, and never answered OK and discarded.	8
38	A GET after a successful SET of the same slot returns the value that was set.	8
39	Slot numbering is append-only, and any renumbering is accompanied by a MAJOR command-set version change.	8.1, 9.3
40	Where one field is reachable through more than one opcode, those opcodes agree on whether it is writable and on the values they accept for it.	8.2

A conforming **host** additionally answers yes to these.

Item	Requirement	See
41	It matches responses by SEQ, and does not assume request ordering.	3.5
42	It tolerates an unsolicited frame arriving at any time, including between its own request and the matching response.	2, 7
43	It tolerates an unrecognised status byte without losing frame synchronisation.	4.2
44	It reads an unsolicited frame's first payload byte as the status byte, and takes the frame's content from DATA[0].	7
45	For a frame that arrived corrupt it tolerates both answers: silence from a device, and [ERR][ERR_BAD_CRC] from a bridge or a bootloader.	3.7
46	It does not send a frame larger than the device's documented maximum, and does not establish that maximum by experiment.	3.8
47	It reads the device's identity and command-set version before using any identifier map, and uses that product's own map.	8.1

Celfras Standard Protocol — Part II: The Products

Part I is the standard. This part is the record of what each product did with the latitude Part I leaves it: the identifier maps behind §8, the values of the constants §3.8 and §5.3 declare product-defined, which optional command groups a product implements, and where a product's behaviour departs from what Part I asks for.

Nothing here relaxes anything in Part I, and nothing here is required of a new implementation. An appendix states facts about one product at one moment. Where such a fact turns out to be needed by *every* implementer, it belongs in Part I and is moved there rather than cited from here.

What this part carries, and what it does not. It carries structure: identifiers, names, units, access, and what each slot is for. It does not carry the numbers a product writes into them. A standard defines what identifier 14 means; what one product stores there is that product's business, and it is carried in this document's internal edition and in the product's own documentation. Where that edition adds a column giving each slot's built-in value, this one simply does not have it, and no threshold value appears anywhere in this text.

That rule is about the contents of the identifier maps. It is not a rule against every number: where Part I requires a device to publish a constant of its own so that a host can talk to it at all — the maximum encoded frame of §3.8, the link-liveness interval of §5.3 — the value is stated here, because a host cannot use the link without it. Read a bare number in this part as one of those link constants, never as a threshold out of a map.

Where this part discusses one slot it names the slot and leaves the number to the table above it. The tables are generated from the products themselves; an id retyped into a sentence is a second copy of the same fact, and this part has already outlived one renumbering (A.2).

A. CFS-ECIG-SUITE

The reference implementation of Part I, and the product every generated identifier table in this document is extracted from.

A.1 What this product implements

Product constants. Part I defers two of these to Part II:

- The largest accepted encoded frame, delimiter excluded (§3.8), is **32 bytes** on the shipping CWM2032 build. The accumulator is sized per chip within the same firmware, so another chip's build of it is a different number; a host talks to a board, not to a source tree, and this is the board's.
- The link-liveness interval, `ONLINE_TIMEOUT_MS` (§5.3), is **5000 ms**.

Optional groups. It implements text logging (§7.1), binary telemetry (§7.2), tuning (§6.4), and both the variable and the parameter family in all three widths (§8). It does **not** implement register access (§6.3) or display (§6.5): those opcodes are declared in its header but nothing dispatches them, so they are answered per §9.1 like any other opcode the device does not act on.

The §6.2 deviation is this product's. `CMD_ACTIVATE`, `CMD_DEACTIVATE`, `CMD_START_HEATING` and `CMD_STOP_HEATING` are the stubs Part I §6.2 names: they answer [OK] whether or not anything happens. An [OK] from those four opcodes is not evidence that the device acted.

`CMD_SET_MODE` also drives the log parameter here. Entering `OPMODE_NORMAL` or `OPMODE_DEBUG` forces the `LOG_ENABLE` parameter slot of A.3 off and on respectively, so the first log line after entering `OPMODE_DEBUG` arrives without a `CMD_LOG_START`. This was a deliberate choice made 2026-07-10, and it is the one behaviour Appendix B contrasts with.

A.2 The variable map

The 8-bit space:

ID	Name	Unit
0	EXT_CTRL_STATUS	EXT_CTRL_* bits
1	EXT_HEATING_PWM_DUTY	%
2	OP_MODE	OPMODE_*
3	STATE	STATE_E
4	OIL_MODE	heating profile idx
5	COIL_SEL	H_COIL*_MASK
6	EVENT_FLAG	EVENT_MASK_*
7	EVENT_TOGGLE_FLAG	EVENT_MASK_*
8	BATTERY_GUAGE	%
9	OIL_GUAGE	%

ID	Name	Unit
10	EVENT_EMUL_MASK	EVENT_MASK_*
11	EVENT_EMUL_ENABLE	0/1

The 16-bit space:

ID	Name	Unit
0	EXT_HEATING_POWER	mW (CP) / mV target_vatrms (CV)
1	PROT_TYPE	PROTECT_MASK_*
2	CHARGING_TIME	min
3	VAT	mV
4	IAT0	mA
5	PAT0	mW
6	RAT0	mohm
7	DUTY0	%

The 32-bit space:

ID	Name	Unit
0	STATE_TIME	ms
1	DISP_TIME	ms
2	ONLINE_ELAPSED	ms
3	HEATING_TIME	ms
4	LED_DAISY_STARTED	bursts
5	LED_DAISY_DONE	bursts

Most of this map is what §8 describes as a read-only view of live firmware state: the state machine's current state and its timers, the measurements the device is taking, the gauges it is computing. A SET to any of those answers ERR_BAD_ARGS.

Five slots are writable, and every one of them is an **input** the host supplies in place of something the firmware would otherwise read off its own hardware — never a second copy of the device's control state. The production control code then runs unmodified rather than growing host-only branches.

- EXT_CTRL_STATUS, EXT_HEATING_PWM_DUTY and EXT_HEATING_POWER are an external-control interface; what its bits mean, what each slot accepts, and what a host must do while driving it are in the product's own documentation, not here.
- EVENT_EMUL_MASK and EVENT_EMUL_ENABLE let a host stand in for the device's own event inputs. Both accept the full width of their slot.

Three further facts a host needs before it trusts a reading:

- **The 16-bit map's measurement half assumes one heating coil.** The per-coil slots are named for coil 0 because this board has only that one; a two-coil board appends its coil-1 slots after them rather than renumbering these.
- **A slot may read zero rather than a measurement, depending on the build.** RAT0 has no current-sense channel to read on a constant-voltage build, and DISP_TIME has no per-state timeline on a daisychain display build. Zero there means *not wired in this build*, exactly as it does for an unwired telemetry bit (§7.2); it is not a reading of zero.
- **The variable numbering happens to be the same on the boards built so far, and that is a coincidence rather than a guarantee.** §8.1 applies to this map exactly as it applies to the parameter map: the agreement is an accident of history, it promises nothing about the next board, and it makes nothing safe. A host still establishes which product it is talking to before it uses either map.

This map was renumbered once, on 2026-07-29, when the external-control block was inserted at the front of the 8-bit and 16-bit spaces instead of being appended. That predates the command-set numbering of Appendix E, so no version difference marks it; the append-only rule of §8.1 has held since, and an older copy of the map addresses the wrong slots.

A.3 The parameter map

The 8-bit space:

ID	Name	Unit	Access
0	LOG_ENABLE	0/1	rw

The 16-bit space:

ID	Name	Unit	Access
0	VDD	mV	ro
1	VAT0	mV	ro
2	IAT0	mA	ro
3	LED_BREATH_PERIOD_MS	ms	rw
4	UVLO_HEAT_INIT_TH	mV	rw
5	UVLO_HEAT_TH	mV	rw
6	LONG_PUFF_TH	ms	rw
7	CHG_OVP_TH	mV	rw
8	CHG_TIMEOUT_TH	min	rw
9	SHORT_COIL1_TH	mV	rw
10	SHORT_COIL2_TH	mV	ro
11	SHORT_COIL_RST_TH	mV	rw
12	OPEN_COIL1_TH	mV	rw
13	OPEN_COIL2_TH	mV	ro

ID	Name	Unit	Access
14	DRY_PUFF_ABS_TEMP_TH	degC	rw
15	DRY_PUFF_REL_SLOPE_TH	degC/sec	rw

The 32-bit space:

ID	Name	Unit	Access
0	HEATING_TIMEOUT_TH	ms	rw

These numbers are this product's, and they are not portable. This is the most consequential fact in Part II, and §8.1 states the rule behind it. Every board numbers its own parameter enumeration independently, so the same id addresses a different field on a different product — and nothing detects it. A host pointed at the wrong board reads the wrong measurement and, worse, writes the wrong threshold: the firmware performs that write and answers **OK**. There is no error at any layer, no checksum that disagrees, and nothing in a log to find afterwards. The only defence is the one §8.1 requires — read the identity and the command-set version first, and use that product's own map.

Most writable slots are open only in a tuning build. The protection thresholds — every rw slot in the 16-bit and 32-bit maps except LED_BREATH_PERIOD_MS — write the device's parameter block, and that block is const in a release build. There, each of them answers ERR_BAD_ARGS instead of storing. That is the intended behaviour of a shipping unit, not a fault and not a regression: a release build is one in which the thresholds are what the product was calibrated to. The tables above give each slot's access in the tuning build, which is the one whose map is worth documenting. Two writable slots are unaffected because they do not touch the parameter block at all: LOG_ENABLE and LED_BREATH_PERIOD_MS route through a service call and stay writable in every build.

Two slots are reserved for a second heating coil this board does not have. SHORT_COIL2_TH and OPEN_COIL2_TH are declared so that the numbering already matches a two-coil build. They read zero and reject every write, in every build — that is not the tuning-build effect above, and selecting a tuning build does not open them.

Four smaller facts about individual slots:

- SHORT_COIL1_TH and OPEN_COIL1_TH are a pair, and the firmware compares them against each other rather than each against a fixed bound: both are tested against the same measured drop, the open-coil fault below one threshold and the short-coil fault above the other. Their ordering is therefore a constraint on what a host may write — the short-coil threshold **MUST** stay above the open-coil one. Crossed over, the healthy band between them becomes a band in which both faults trip at once, so crossing them disables coil detection instead of tightening it.
- CHG_OVP_TH and SHORT_COIL_RST_TH are readable and writable, but nothing currently compares against them — one is a divider reading whose compare is disabled, the other belongs to a protection detected by other means. They are kept for numbering stability, and writing them changes nothing today.
- LED_BREATH_PERIOD_MS **accepts a zero it does not store**: zero would divide by zero in the display ramp, so the setter returns early while the command still answers [OK], and a GET

afterwards returns the previous value. That is a departure from §8's read-back rule, recorded here rather than smoothed over. Its accepted range is one and above, and a new implementation MUST answer ERR_BAD_ARGS for the zero rather than copy this.

- LOG_ENABLE **stores something other than what it was given**: its setter reduces any non-zero value to one, and the command answers [OK] regardless. A SET of any non-zero value is therefore answered OK, and a GET afterwards returns one. That is the masking §8 forbids, and it is the same defect as the slot above seen from the other side — a setter with no way to report a refusal to the layer that composes the response, so the response says OK. Its accepted range is zero and one, and a new implementation MUST answer ERR_BAD_ARGS for anything else rather than copy this. The two are one defect with two instances, and are recorded on this product's own defect list as one entry rather than as two unrelated quirks.

Apart from those two, a writable parameter slot accepts the full width of its slot: the firmware range-checks the identifier, not the value. Where a value has to be sane for the device to behave, that constraint is the product's, and it is in the product's own documentation.

A.4 Telemetry fields

Part I §7.2 fixes which bit is which quantity and leaves two things to the product: the scale behind each field, and which bits are wired to a real reading at all. On this product every field is a u16, and the scales are:

- LOG_FIELD_VDD, LOG_FIELD_VAT — millivolts.
- LOG_FIELD_IAT — milliamperes.
- LOG_FIELD_PWR — milliwatts, derived from the measured load voltage and current and the drive's duty cycle rather than measured directly.
- LOG_FIELD_DUTY — percent.
- LOG_FIELD_RAT — milliohms, computed by the device rather than measured.
- LOG_FIELD_PROT — a bitfield of the device's protection status, not a scaled quantity.

Wired to a real reading: VDD, VAT, IAT, PWR, DUTY, and RAT on a constant-power build with dry-puff support. PROT is not wired, and RAT reads zero on a constant-voltage build, which has no current-sense channel to compute it from. An unwired bit is still accepted and still occupies its u16 in the frame, reporting zero, exactly as §7.2 requires.

A.5 Tuning identifiers

Part I §6.4 fixes the session shape and leaves the identifiers to the product. This one defines four, and CMD_TUNING_START refuses an identifier the running build does not support with ERR_BAD_ARGS:

- TUNING_ID_CV_VATRMS (0) — sweeps the constant-voltage drive target. Constant-voltage builds only.
- TUNING_ID_BATTERY_GAUGE (1) — sweeps the battery gauge's voltage divisions. It is defined over a discharge phase and a charge phase, and **the shipping firmware runs only the charge phase**. Which phase a session takes is decided by the device, from whether a charger is actually present; with none present the session would be a discharge sweep, and CMD_TUNING_START refuses the identifier outright with ERR_NOT_READY. The refusal is deliberate: the discharge sweep's frames

would report the voltage under load rather than the cell's own, so the sweep would return numbers that look like a calibration and are not, and refusing is the honest answer. The cause is a regression in this product's measurement path and is recorded on its own defect list. This refusal is not the staging refusal below — it does not depend on anything the host staged — and it shares only its error code.

- TUNING_ID_PROTECT_THRESH (2) — sweeps the short-coil and open-coil resistance thresholds.
- TUNING_ID_DRYPUFF (3) — sweeps the dry-puff temperature and slope thresholds. Constant-power builds with dry-puff support only.

A session runs the real production control algorithm through the same external-control input path as A.2, so a host stages the stimulus before starting one; the staging belongs with that interface, in the product's own documentation. CMD_TUNING_START refuses a session whose staged stimulus could produce no measurement, with ERR_NOT_READY, rather than running to completion and reporting nothing.

One identifier also constrains the requested report period. A battery-gauge session reports once per cycle of its own fixed sample cadence, so the period has nothing left to control: CMD_TUNING_START refuses any other value with ERR_BAD_ARGS instead of accepting it and reporting at a different rate, since a host that asked for one period and silently got another would misread every gap between timestamps. The cadence itself is a constant of this product and is stated with the product's own documentation. No other identifier constrains the period beyond what §6.4 requires.

What the index tuple means. §6.4 leaves the meaning of CMD_SET_TUNING_PAR/CMD_GET_TUNING_PAR's indices to the product. Here the tuning_id selects the calibration table and the remaining indices address a cell in it; an identifier uses only the indices its own table has dimensions for, and every index it uses is bounds-checked against the real dimension, a violation answering ERR_BAD_ARGS as §6.4 requires. No session is needed for either command.

- TUNING_ID_CV_VATRMS is the only one that uses the whole tuple: coil_idx the heating coil, heating_mode the drive mode, heater_type the fitted heater, step the target-voltage step.
- TUNING_ID_BATTERY_GAUGE uses coil_idx as the **charge direction** — discharging or charging, not a physical coil — and step as the gauge division.
- TUNING_ID_PROTECT_THRESH uses coil_idx as the heating coil and step to select which of the pair is addressed, the short-coil threshold or the open-coil one.
- TUNING_ID_DRYPUFF uses step to select the absolute-temperature threshold or the slope threshold; these are not per-coil, but coil_idx is still checked against the coil count, so a host naming a coil that does not exist is told rather than served.

An index an identifier does not use is ignored rather than required to be zero, and is not range-checked. A host SHOULD send zero for it, because a product that later gives that dimension a meaning will start checking it.

Which protections a session suspends — the disclosure §6.4 requires: long-puff and heating-timeout protection are suspended whenever the host is driving the heating trigger, which covers plain external control as well as a tuning session; charge-timeout protection is suspended for the duration of a battery-gauge session, because charging across the cell's range is that measurement; and the coil-resistance check is suspended for a protect-threshold session, which is the sweep that has to reach the

values the check exists to refuse. Under-voltage lockout and charge over-voltage protection stay live throughout. A host that relied on the long-puff or heating-timeout protection to end an over-long externally driven heating request does not have that backstop.

One of those suspensions does not reach as far as it reads. Suspending the coil-resistance check suspends the check the device runs *while* heating, and not the one it runs at heating *start*: that one raises an open-coil fault directly from a pin reading, with nothing in its path that the suspension touches. So a protect-threshold session on a load the check would refuse can still be stopped at heating start — the exact case the suspension exists to permit. It is recorded on this product's own defect list, and a product that discloses a suspension **SHOULD** make it reach every check the disclosure covers.

One inconsistency on this product's protocol surface is worth stating, because it is host-visible: on a constant-voltage tuning build the two dry-puff threshold slots are writable through `CMD_PAR16_SET`, which answers OK, and refused by `CMD_SET_TUNING_PAR`, which answers `ERR_BAD_ARGS` on the correct reasoning that nothing reads them in that build. Two opcodes, one field, opposite answers. That is a departure from §8.2, which requires two opcodes reaching one field to agree about whether it is writable. It is a protocol-surface inconsistency only — the field exists either way — and it is recorded, not yet resolved.

B. CVS-BP2601

A second product on the same standard. Its identifier maps are its own and are held with that product; §8.1 applies to them in full, and Appendix A must not be read as describing them.

Exactly one difference from Appendix A is on record, and it is the one A.1 sets up. **`CMD_SET_MODE` on CVS-BP2601 does not touch the log-enable parameter.** Entering `OPMODE_DEBUG` there leaves logging off until an explicit `CMD_LOG_START`, where CFS-ECIG-SUITE forces it off and on with the mode. Both behaviours conform — §5 defines what a mode is for and does not tie a parameter slot to it — so a host that wants log output from either product asks for it explicitly rather than inferring it from the mode it selected. The CFS-ECIG-SUITE behaviour was a deliberate choice made in that project on 2026-07-10 and has not been back-ported.

C. The bridge layer

The bridge is the relay of §1.3, and §9.2 already states the rule from the device's side: `0xC0` and above belongs to this layer, a product must not define a command there, and a bridge routes on the opcode byte alone. This appendix is the table that rule reserves the band for.

ID	Name
0xC0	CMD_B_PING
0xC1	CMD_B_INFO
0xC2	CMD_B_GET_CAPS

ID	Name
0xC3	CMD_B_GET_STATS
0xC4	CMD_B_GET_SNAPSHOT
0xC5	CMD_SET_SWD_SPEED
0xC6	CMD_SET_TARGET
0xC7	CMD_B_GET_VERSION
0xC8	CMD_MODE_DEBUG
0xC9	CMD_MODE_DOWNLOAD
0xCA	CMD_MODE_RAW
0xCB	CMD_MODE_BRIDGE_RESET
0xCC	CMD_TGT_NRST
0xCD	CMD_TGT_POWER
0xCE	CMD_BRIDGE_CONF
0xD0	CMD_SWD_CONNECT
0xD1	CMD_SWD_DISCONNECT
0xD2	CMD_SWD_LINE_RESET
0xD3	CMD_SWD_JTAG2SWD
0xD4	CMD_SWD_READ_IDCODE
0xD7	CMD_SWD_CONNECT_UNDER_RESET
0xD8	CMD_DP_READ
0xD9	CMD_DP_WRITE
0xDA	CMD_AP_READ
0xDB	CMD_AP_WRITE
0xE0	CMD_CORE_HALT
0xE1	CMD_CORE_RESUME
0xE2	CMD_CORE_STATUS
0xE3	CMD_CORE_SYSRESET
0xE4	CMD_CORE_RUN_DETACH
0xE8	CMD_UPD_BEGIN
0xE9	CMD_UPD_DATA
0xEA	CMD_UPD_END
0xEB	CMD_UPD_STATUS
0xEC	CMD_UPD_ABORT
0xED	CMD_UPD_APPLY
0xF0	CMD_FLASH_ERASE_4K

ID	Name
0xF1	CMD_FLASH_PROG_PAGE_128
0xF2	CMD_FLASH_PROG_PAGE_128_VERIFY
0xF3	CMD_FLASH_READ
0xF4	CMD_FLASH_MASS_ERASE
0xF5	CMD_FLASH_READ_EX
0xF8	CMD_OB_READ
0xF9	CMD_OB_WRITE
0xFA	CMD_OB_LAUNCH
0xFB	CMD_OB_WRITE_AND_LAUNCH

Read it as one layer's table, not one board's. Two bridge boards are in service; they differ in performance, not in what they answer, and both declare this table. That is what makes §9.2's reservation a reservation rather than a per-board convention.

Four things a host should know before using it:

- **Some entries are reserved without a handler**, and the table does not mark which. Whether an opcode has one is a property of the image that answers, not of the band's table: it changes with the firmware version, and a bootloader answers far fewer opcodes than an application. A reserved opcode is answered ERR_BAD_ARGS like any other unimplemented one (§9.1), so a host probes for one rather than reading a version first.
- **CMD_UPD_* (0xE8-0xED) update the bridge itself, and only its bootloader answers them.** They are not the CMD_FLASH_* block, which programs the *device* behind the bridge. Confusing the two is a mistake with no protocol-level defence: both blocks are in this same band and both answer plausibly. Their shapes are: CMD_UPD_BEGIN takes a fixed 512-byte image header and answers [OK][chunk_max u16][capacity u32]; CMD_UPD_DATA takes [off u32][data], at most 256 data bytes and never more than the chunk_max just returned, and answers [OK][next_off u32]; CMD_UPD_STATUS takes no payload and answers [OK] followed by a fixed 18-byte status block, whose field layout is the bridge's own and is documented with it; and CMD_UPD_END, CMD_UPD_ABORT and CMD_UPD_APPLY take no payload and answer [OK], the board copying the image and resetting itself after it has answered CMD_UPD_APPLY. Those sizes are wire constants of this block, the same on every bridge board.
- **CMD_BRIDGE_CONF is one opcode for many settings**, keyed by a parameter byte, because the band has few free numbers and a toggle should not consume one. An unknown parameter is answered ERR_BAD_ARGS, which is how a host detects support for a setting without reading a version.
- **CMD_B_PING and CMD_B_INFO exist because the device's own CMD_PING and CMD_INFO cannot say who answered.** Both layers answer those the same way, so a host confirming that the *bridge* is there asks in the bridge's own band.

The bridge's version command carries its own selector space, as §6.1 warns: the same selector number means something different from the device's. Besides a firmware version it answers the running image's

role, which distinguishes the application from the bootloader — the first thing a host should ask, since a firmware version alone does not say which image produced it — and the board's **hardware identity**, which is the one selector whose answer must be the same from either image, because the hardware does not change when the image does.

The role selector is also where a host learns what to expect from the command-set selector. A bridge bootloader answers that selector with its own number, deliberately frozen below the application's: it names the last revision that changed the handful of opcodes the bootloader itself implements, and it does not move when the application's number does. A host that has just read the role and found a bootloader should expect the next command-set version it reads to be lower, and **MUST NOT** treat the difference as a fault or as a mis-built release. §9.3's release-bookkeeping rule is about the copies of one image's constant, and the bootloader is a different image.

The bridge's error codes live in the extension space of §4.4, which names their blocks. One shape is worth adding here: every `ERR_UPD_*` code carries a u32 detail after the code byte, because the useful question on a bench is not whether an update failed but at which offset, which address, or which check.

D. DUT test firmwares

Three small firmwares exist so that a bridge always has a conforming device to talk to, one for each chip a bridge is used against: **CWM2032**, **CWM1016** and **CWM0508**. They are test targets, not products, and ship in nothing.

Each implements exactly the six opcodes §5.2 requires to be answered in `OPMODE_NORMAL` — `CMD_PING`, `CMD_INFO`, `CMD_GET_VERSION`, `CMD_SET_MODE`, `CMD_DBG_ONLINE` and `CMD_RESET` — and nothing else, and each declares the same command-set version as the products do, for the reason §9.3 gives: the version names the opcode table, not the size of the implementation behind it. A device that implements six opcodes and answers the rest per §9.1 is as conforming as one that implements them all.

E. Command-set change history

One version number covers both bands (§9.3). It is duplicated across every place in this list:

- the CFS-ECIG-SUITE firmware, and its host tool;
- each of the two bridge firmwares, each of those two boards' build-metadata files, and the bridge host tool;
- each of the three test firmwares of Appendix D.

None of them is authoritative over the others; a release in which they disagree is mis-built, which is the release-bookkeeping rule §9.3 states. The list is the fact, and no total is written beside it: §9.3 says these copies have to be counted rather than assumed, and a total written next to a list is a second thing to keep true — it is the one that rots, and it would rot again the next time a board is added. Count the list.

Two things carry the same constant and are deliberately not in it. A frozen reference tree carries it because it is a comparison snapshot, not a product. The bridge bootloaders carry a number of their

own, frozen and lower, for the reason Appendix C gives.

Every entry below is derived from the history of those copies rather than from a changelog kept for the purpose, and each names what changed on the wire.

- **2.0.0** (2026-08-21) — the number itself. Both layers began declaring a command-set version together. `CMD_GET_VERSION` (0x05) was added on the device side and `CMD_B_GET_VERSION` (0xC7) on the bridge side, and the bridge's two configuration commands moved below its SWD gate, where they belong, since neither drives an SWD pin.
- **2.1.0** (2026-08-26) — `CMD_FLASH_READ` (0xF3) went from reserved to implemented.
- **2.2.0** (2026-08-26) — the bridge self-update block `CMD_UPD_*` (0xE8-0xED), bootloader-only, with its `ERR_UPD_*` error codes; a version selector for the running image's role; and `CMD_MODE_BRIDGE_RESET` (0xCB) from reserved to implemented.
- **2.3.0** (2026-08-27) — `CMD_BRIDGE_CONF` (0xCE), one opcode carrying runtime settings as a parameter/value pair instead of an opcode per setting.
- **2.4.0** (2026-08-30) — a hardware-identity selector on the bridge's version command, answering board identity and revision from either image. Until then the only way to tell two bridge boards apart was free text.
- **2.5.0** (2026-08-30) — core halt and core resume were given their own error codes instead of borrowing a flash one, which had cost bench time (§4.4).
- **2.6.0** (2026-09-01) — an option-byte mask bit named in the bridge's option-byte commands.
- **2.7.0** (2026-09-03) — a bridge configuration key for working with a target whose reset line is not wired.
- **2.8.0** (2026-09-04) — a bridge configuration key selecting the SWD pin swap.
- **2.9.0** (2026-09-04) — the pin-map configuration key, the first whose value carries meaning rather than being a flag, together with the host-side fix that stopped flattening configuration values to zero or one — which would have sent that key's second value as its first. The bridge would then have selected the first map and echoed back the value it had actually received, so the reply agreed with the frame that arrived, and nothing on the wire would have shown that the host had asked for the other map.
- **2.10.0** (2026-09-08) — `CMD_FLASH_READ_EX` (0xF5), a read that also returns the fault count the plain read cannot report.
- **2.11.0** (2026-09-09) — current. Writes to `CMD_TGT_POWER` were placed behind a bridge configuration key, so a board able to supply its target's power does not do so by default.

Where the record runs out. It runs out below 2.0.0, and there is nothing to recover: the command-set version was introduced on 2026-08-21 already numbered 2.0.0, and no implementation ever declared a 1.x. The v1.x numbers that appear in the reference firmware's own protocol document are that document's own revision numbers, and they are not command-set versions; reading them as earlier entries in this list would be a mistake. They are not empty, though. That document bumps its number when a wire-visible change lands — a command added, removed or renumbered, a frame layout changed, a default behaviour changed — and not for internal implementation changes, and its v1.0 through v1.5 entries record real ones: a log opcode moved, the burst payload was reordered, an operating mode was retired, the default mode's answer to an unknown command changed from an error to silence, and a tuning command's payload grew a leading field. They describe wire changes on one product that predate the command-set numbering entirely.

Two properties of the list are worth stating plainly. Every bump since 2.0.0 has been MINOR, and every one of them landed in the bridge band: the device band has not changed since 2.0.0, which is why a device built against 2.0.0 and a host built against 2.11.0 still agree about every opcode the device has. That is a statement about the numbered range and about nothing earlier — the device band was renumbered repeatedly in the weeks before the numbering began, as the v1.x entries above record, which is part of why the numbering exists. And not every bump is a wire change — 2.9.0 is partly host-side bookkeeping — because the rule of §9.3 is that every copy in the list above moves together, not that each increment adds an opcode.